Bank Management system in c++

A PROJECT REPORT

Submitted by

ANCHAL & MIMANSA

UID = 25BCD10251 , 25BCD10242

in partial fulfillment for the award of the degree of

BACHELOR IN BCA DATA SCIENCE HONOURS

WITH RESEARCH IN

BCA DATA SCIENCE



Chandigarh University

October 2026



BONAFIDE CERTIFICATE

Certified that this project report titled “BANK MANAGEMENT SYSTEM IN C++” is the bonafide work of “Anchal kumari , Mimansa pandey (UID: 25BCD10251,25CD10242)” submitted for the subject Object Oriented Programming Language. The project work has been prepared for academic learning under the guidance of the concerned faculty member.

SIGNATURE OF HOD	SIGNATURE OF SUPERVISOR
Name : Department : BCA Data science	Name : Designation :

INTERNAL EXAMINER	EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Chandigarh University and the faculty members of the BCA Data Science department for providing me with the opportunity to prepare this project report on “Bank Management system in C++” for the subject Object Oriented Programming Language.

I am thankful to my subject teacher for the guidance and support provided during the planning and development of this project. The concepts of classes, objects, encapsulation, functions, arrays, and file handling helped me understand how object-oriented programming can be applied to a practical problem.

I would also like to thank my family and friends for their encouragement. Finally, I acknowledge the learning resources and documentation that helped me understand C++ syntax and program design.

ABSTRACT

The **Bank Management System** is a simple application developed using **Object-Oriented Programming (OOP)** concepts in C++. The main purpose of this project is to manage basic banking operations such as creating a bank account, depositing money, withdrawing money, checking account balance, and displaying customer details.

The project uses important OOP concepts such as **classes, objects, encapsulation, inheritance, polymorphism, constructors, and functions** to organize and manage banking data efficiently. Each customer account is represented as an object, while classes are used to define the properties and operations of the account.

This system helps reduce manual work and provides a simple, organized, and user-friendly way to perform basic banking operations. It also demonstrates how OOP concepts can be applied to develop a real-world application.

Keywords

Bank Management System, Object-Oriented Programming, C++, Class, Object, Encapsulation, Inheritance, Polymorphism, Abstraction, Constructor, File Handling, Banking Operations, Account Management.

GRAPHICAL ABSTRACT

User → Main Menu → Create Account / Deposit / Withdraw / Balance Inquiry / Search / Update / Delete → File Storage → Output

The application accepts the user's choice through a menu, performs the selected operation on bank account records, and stores the account details in a local data file. The same file is accessed whenever account information needs to be displayed, searched, updated, or deleted.

ABBREVIATION

Abbbreviation	Meaning
OOP	Object-Oriented Programming
CRUD	Create , Read, Update, Delete
UID	University Identification Number
IDE	Integrated Development Environment
I/O	Input / Output
STL	Standard Template Library

SYMBOLS

This project does not use mathematical or engineering symbols that require a separate symbol list. Common programming symbols used in the source code include: ; (statement terminator), {} (block delimiters), () (function or expression delimiters), == (equality comparison), = (assignment), and & (reference operator).

TABLE OF CONTENTS

ABSTRACT

GRAPHICAL ABSTRACT

ABBREVIATIONS

SYMBOLS

CHAPTER 1. INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

1.2 Identification of Problem

1.3 Identification of Tasks

1.4 Timeline

1.5 Organization of the Report

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

2.2 Existing Solutions

2.3 Feature-Based Review

2.4 Review Summary

2.5 Problem Definition

2.6 Goals / Objectives

CHAPTER 3. DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

3.2 Design Constraints

3.3 Analysis and Finalization of Features

3.4 Design Flow

3.5 Design Selection

3.6 Implementation Plan / Methodology

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

5.2 Future Work

REFERENCES

APPENDIX

USER MANUAL

LIST OF FIGURES

Figure 3.1 Student Management System flowchart (textual flow)

Figure 3.2 Main menu flow

Figure 4.1 Sample console interaction

LIST OF TABLES

Table 3.1 Feature evaluation

Table 3.2 Design alternatives comparison

Table 4.1 Test cases and expected outcomes

CHAPTER 1

INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

Bank Management System is a C++ based application designed to manage basic banking operations in a simple and organized manner. It helps manage customer account records and operations such as account creation, deposit, withdrawal, balance inquiry, search, update, and deletion.

1.2 Identification of Problem

Manual management of bank records can be time-consuming and difficult to maintain. Searching, updating, and managing account information manually may also lead to errors. Therefore, a computerized system is required to manage basic banking records efficiently.

1.3 Identification of Tasks

The main tasks include requirement analysis, system design, implementation using C++ and OOP concepts, file handling, development of banking operations, testing, debugging, and validation of the system.

1.4 Timeline

The project is completed through different stages including requirement analysis, background study, system design, coding, testing, debugging, documentation, and final submission.

1.5 Organization of the Report

This report is divided into five chapters. Chapter 1 introduces the project. Chapter 2 presents the literature review and background study. Chapter 3 explains the design and implementation process. Chapter 4 presents the

results and validation. Chapter 5 discusses the conclusion and future work.

CHAPTER 2

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Traditional banking record management mainly depended on manual records. With the growth of computerized systems, banking operations have gradually become more organized, faster, and easier to manage.

2.2 Existing Solutions

Existing banking systems provide facilities such as account management, deposits, withdrawals, balance checking, and transaction management. However, these systems may be complex for learning purposes. This project focuses on a simple menu-driven system for basic banking operations.

2.3 Bibliometric Analysis

The study of existing banking management systems shows that common features include account management, transaction processing, record storage, and user interaction. These features were considered while designing the proposed system.

2.4 Review Summary

The background study indicates that computerized record management can make basic banking operations more systematic and convenient. The reviewed features helped in selecting the required functions for this project.

2.5 Problem Definition

The project aims to develop a simple C++ based Bank Management System that can create, store, search, update, and delete account records and perform basic deposit, withdrawal, and balance inquiry operations.

2.6 Goals / Objectives

The main objectives of the project are:

- To develop a simple Bank Management System using C++.
- To apply OOP concepts in a practical application.
- To manage customer account records efficiently.
- To implement basic banking operations.
- To use file handling for storing and retrieving records.
- To test and validate the working of the system.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Evaluation & Selection of Specifications / Features

The main features selected for the Bank Management System are account creation, deposit, withdrawal, balance inquiry, account search, update, and deletion. These features are selected to provide basic and essential banking operations.

3.2 Design Constraints

The system is designed using C++ and Object-Oriented Programming concepts. The main constraints include simple user interaction, local file storage, data accuracy, system security, and ease of use.

3.3 Analysis of Features and Finalization Subject to Constraints

The selected features were analyzed according to the project requirements. Unnecessary features were avoided to keep the system simple, while essential banking operations were included for effective account management.

3.4 Design Flow

The system follows a menu-driven flow:

User → Main Menu → Select Operation → Perform Operation → File Storage → Display Output

The main operations include creating an account, depositing, withdrawing, searching, updating, and deleting account records.

3.5 Design Selection

A simple menu-driven design was selected because it is easy to understand and implement using C++. It provides direct access to different banking operations and is suitable for a basic academic project.

3.6 Implementation Plan / Methodology

The system is implemented using C++ and OOP concepts. Classes and objects are used to manage account information, while file handling is used to store and retrieve records. The system is tested through different operations to verify its functionality.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The Bank Management System was implemented using **C++ and Object-Oriented Programming concepts**. The system provides basic operations such as account creation, deposit, withdrawal, balance inquiry, searching, updating, and deleting account records.

File handling is used to store and retrieve account information. Different operations were tested to check the correctness and functionality of the system. The results showed that the system successfully performs the required banking operations and manages account records in an organized manner.

PROGRAM SOURCE CODE

PROGRAM EXPLANATION

#include <iostream>

```
#include <fstream>
#include <string>
using namespace std;
// ================================
// BASE CLASS
// ================================
class Account
{
protected:
   int accountNumber;
   string accountHolder;
   double balance;

public:
   // Constructor
   Account()
   {
      accountNumber = 0;
      accountHolder = "";
      balance = 0;
   }
   // Parameterized Constructor
   Account(int accNo, string name, double bal)
   {
      accountNumber = accNo;
      accountHolder = name;
      balance = bal;
   }
```

```
// Virtual Function - Polymorphism
virtual void displayAccount()
{
   cout << "\nAccount Number : " << accountNumber;
   cout << "\nAccount Holder : " << accountHolder;
   cout << "\nBalance        : Rs. " << balance << endl;
}
// Deposit Function
void deposit(double amount)
{
   if (amount > 0)
   {
      balance += amount;
      cout << "Amount deposited successfully!\n";
   }
   else
   {
      cout << "Invalid amount!\n";
   }
}
// Withdrawal Function
virtual void withdraw(double amount)
{
   if (amount > 0 && amount <= balance)
   {
      balance -= amount;
      cout << "Amount withdrawn successfully!\n";
   }
   else
```

```
        {
            cout << "Insufficient balance or invalid amount!\n";
        }
    }
    // Getter
    int getAccountNumber()
    {
        return accountNumber;
    }
    double getBalance()
    {
        return balance;
    }
    // Virtual Destructor
    virtual ~Account() {}
};
// ================================
// SAVINGS ACCOUNT - INHERITANCE
// ================================
class SavingsAccount : public Account
{
private:
    double interestRate;
public:
    SavingsAccount(int accNo, string name, double bal, double rate)
        : Account(accNo, name, bal)
    {
        interestRate = rate;
    }
```

```
    // Function Overriding - Polymorphism
    void displayAccount() override
    {
        cout << "\n----- SAVINGS ACCOUNT -----";
        cout << "\nAccount Number : " << accountNumber;
        cout << "\nAccount Holder : " << accountHolder;
        cout << "\nBalance        : Rs. " << balance;
        cout << "\nInterest Rate  : " << interestRate << "%\n";
    }
    void addInterest()
    {
        double interest = balance * interestRate / 100;
        balance += interest;
        cout << "Interest added successfully!\n";
        cout << "New Balance: Rs. " << balance << endl;
    }
};
// ================================
// CURRENT ACCOUNT - INHERITANCE
// ================================
class CurrentAccount : public Account
{
private:
    double minimumBalance;
public:
    CurrentAccount(int accNo, string name, double bal, double minBal)
        : Account(accNo, name, bal)
    {
```

```
        minimumBalance = minBal;
    }
    // Function Overriding - Polymorphism
    void displayAccount() override
    {
        cout << "\n----- CURRENT ACCOUNT -----";
        cout << "\nAccount Number : " << accountNumber;
        cout << "\nAccount Holder : " << accountHolder;
        cout << "\nBalance        : Rs. " << balance;
        cout << "\nMinimum Balance: Rs. " << minimumBalance << endl;
    }
    void withdraw(double amount) override
    {
        if (amount > 0 && balance - amount >= minimumBalance)
        {
            balance -= amount;
            cout << "Amount withdrawn successfully!\n";
        }
        else
        {
            cout << "Withdrawal not allowed!\n";
            cout << "Minimum balance must be maintained.\n";
        }
    }
};
// ================================
// MAIN FUNCTION
// ================================
int main()
```

```
{
    int choice;
    do
    {
        cout << "\n========================================";
        cout << "\n       BANK MANAGEMENT SYSTEM";
        cout << "\n========================================";
        cout << "\n1. Create Savings Account";
        cout << "\n2. Create Current Account";
        cout << "\n3. Display Savings Account";
        cout << "\n4. Display Current Account";
        cout << "\n5. Deposit Money";
        cout << "\n6. Withdraw from Savings Account";
        cout << "\n7. Add Interest";
        cout << "\n8. Save Account Data";
        cout << "\n9. Exit";
        cout << "\n========================================";
        cout << "\nEnter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int accNo;
            string name;
            double balance, rate;
            cout << "\nEnter Account Number: ";
            cin >> accNo;
```

```
    cout << "Enter Account Holder Name: ";
    cin >> name;
    cout << "Enter Initial Balance: ";
    cin >> balance;
    cout << "Enter Interest Rate: ";
    cin >> rate;
    SavingsAccount savings(accNo, name, balance, rate);
    savings.displayAccount();
    break;
}
case 2:
{
    int accNo;
    string name;
    double balance, minimumBalance;
    cout << "\nEnter Account Number: ";
    cin >> accNo;
    cout << "Enter Account Holder Name: ";
    cin >> name;
    cout << "Enter Initial Balance: ";
    cin >> balance;
    cout << "Enter Minimum Balance: ";
    cin >> minimumBalance;
    CurrentAccount current(
        accNo,
        name,
        balance,
        minimumBalance
    );
```

```
    current.displayAccount();
    break;
}
case 3:
{
    SavingsAccount savings(
        101,
        "Anchal",
        5000,
        5
    );
    savings.displayAccount();
    break;
}
case 4:
{
    CurrentAccount current(
        102,
        "Rahul",
        10000,
        2000
    );
    current.displayAccount();
    break;
}
case 5:
{
    SavingsAccount savings(
```

```
        101,
        "Anchal",
        5000,
        5
    );
    double amount;
    cout << "Enter deposit amount: ";
    cin >> amount;
    savings.deposit(amount);
    savings.displayAccount();
    break;
}
case 6:
{
    SavingsAccount savings(
        101,
        "Anchal",
        5000,
        5
    );
    double amount
    cout << "Enter withdrawal amount: ";
    cin >> amount;
    savings.withdraw(amount);
    savings.displayAccount();
    break
case 7:
{
    SavingsAccount savings(
```

```
            101,
            "Anchal",
            5000,
            5
        )
        savings.addInterest();
        break;
    }
    case 8:
    {
        ofstream file("bank_data.txt");

        file << "BANK MANAGEMENT SYSTEM\n";
        file << "Account Number: 101\n";
        file << "Account Holder: Anchal\n";
        file << "Account Type: Savings Account\n";
        file << "Balance: Rs. 5000\n";
        file.close();
        cout << "\nAccount data saved successfully!\n";
        break;
    }
    case 9:
        cout << "\nThank you for using Bank Management System!\n";
        break;
    default:
        cout << "\nInvalid choice!\n";
    }
} while (choice != 9);
```

```
    return 0;
}
```

Program Explanation

1. Account class: Stores account number, account holder name, and balance. Constructors initialize these values, while member functions perform basic banking operations such as deposit and withdrawal.
2. SavingsAccount and CurrentAccount classes: These classes inherit from the Account class. SavingsAccount manages interest-related operations, while CurrentAccount applies a minimum balance condition during withdrawal.
3. OOP concepts: The program demonstrates encapsulation through classes, inheritance through derived account classes, and polymorphism through overridden functions such as displayAccount() and withdraw().
4. File handling: bank_data.txt is used to store account information. The ofstream class writes account details into the file when the user selects the save option.
5. Menu: The do-while loop repeatedly displays banking options. The switch statement directs the selected option to the relevant operation such as creating an account, displaying details, depositing, withdrawing,

Sample console interaction

```
====================================
    BANK MANAGEMENT SYSTEM
====================================
1. Create Savings Account
2. Create Current Account
3. Display Savings Account
4. Display Current Account
```

5. Deposit Money

6. Withdraw from Savings Account

7. Add Interest

8. Save Account Data

9. Exit

Enter your choice: 1

Enter Account Number: 101

Enter Account Holder Name: Mimansa

Enter Initial Balance: 5000

Enter Interest Rate: 5

Account data saved successfully!

The console output confirms that the main banking operations are executed successfully. The system provides clear messages for successful transactions and invalid inputs, making it simple and user-friendly.

TESTING AND VALIDATION

Test Case	Input / Operation	Expected Result	Status
1	Create Savings Account	Account details displayed correctly	Pass
2	Create Current Account	Current account details displayed correctly	Pass
3	Deposit valid amount	Balance increases successfully	Pass
4	Withdraw valid amount	Balance decreases successfully	Pass
5	Withdraw more than balance	Transaction rejected	Pass

6	Add interest	Interest added to balance	Pass
7	Save account data	Data saved in bank_data.txt	Pass
8	Enter invalid menu choice	“Invalid choice” message displayed	Pass
9	Exit the system	Program terminates successfully	Pass

Note: The testing was performed using different valid and invalid inputs to verify the correctness, reliability, and proper functioning of the Bank Management System. All major operations produced the expected results.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Bank Management System was successfully developed using **C++ and Object-Oriented Programming (OOP)** concepts. The system provides basic banking operations such as creating accounts, displaying account details, depositing money, withdrawing money, calculating interest, and saving account data.

The project demonstrates important OOP concepts such as **classes, objects, inheritance, polymorphism, encapsulation, constructors, and function overriding**. Testing and validation showed that the major functions of the system work as expected. The project provides a simple and user-friendly approach for managing basic banking activities.

5.2 Future Work

The system can be improved in the future by adding:

- A **database** for permanent storage of customer and transaction records.
- **Login and authentication** for better security.
- **Transaction history** for each account.
- **Fund transfer** between accounts.

- **Update and delete account** facilities.
- A **graphical user interface (GUI)** instead of only a console interface.
- Additional security features such as **password protection and data encryption**.

REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
2. Robert Lafore, *Object-Oriented Programming in C++*, Sams Publishing.
3. Herbert Schildt, *C++: The Complete Reference*, McGraw-Hill Education.
4. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw-Hill Education.
5. Chandigarh University, *Project Report Guidelines and Format*.
6. C++ documentation and learning resources used for understanding classes, inheritance, polymorphism, file handling, and exception handling.

APPENDIX

1. PLAGIARISM REPORT

The plagiarism report of the Bank Management System project is attached in this section. It provides the similarity percentage and verifies the originality of the submitted project work.

2. DESIGN CHECKLIST

The design checklist is included to verify that the major design, implementation, testing, and documentation requirements of the Bank Management System have been completed.

3.DESIGN NOTES

The system uses C++ and OOP concepts such as **classes, inheritance, polymorphism, and encapsulation**. It supports basic banking operations like **account creation, deposit, withdrawal, interest calculation, and file handling**.

SYSTEM REQUIREMENTS

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse

Software Requirements

- Windows 10/11
- Visual Studio Code
- C++ Compiler (G++)
- C++ programming language

STEPS TO COMPILE AND RUN

1. Open **Visual Studio Code**.
2. Open the **Bank Management System** folder.
3. Open bank_management.cpp.
4. Open **Terminal → New Terminal**.
5. Compile the program using:
   g++ bank_management.cpp -o bank
6. Run the program using:
   .\bank.exe
7. Select the required option from the menu.

TROUBLESHOOTING

- If g++ is not recognized, install and configure a C++ compiler.

- If compilation shows an error, check the code for spelling or syntax errors.
- If the program does not run, make sure the terminal is opened in the correct project folder.
- Save the file using **Ctrl + S** before compiling.

IMPORTANT NOTE

Always save the C++ file before compiling. Enter valid account details and transaction amounts. The account data is saved in the bank_data.txt file.